

# QuantForge Cumulative User Manual — v26.25

> Current release focus: governed market-data intake, provenance, and fail-closed optimization admission.

## QuantForge User Manual — Cumulative v26.05

Document status: Current stable-build manual

Build: v26.05

Audience: Consumer/research user, advanced researcher, contributor, and technical operator

### 1. Quick start

#### ***What QuantForge is***

QuantForge is a research, strategy-audit, simulation, optimization, replay, and controlled forward-testing platform. It is designed to let users inspect and test strategy behavior without granting live trading authority.

#### ***What you can do in this build***

- Import strategy/indicator source into quarantine.
- Review what the source appears to do.
- Review safety-sensitive capabilities before integration.
- Approve only permitted research features.
- Run causal historical research when valid market data is available.
- Run walk-forward and replay research.
- Run simulated forward testing.
- Run multiple research strategies with isolated state and ledgers.
- Review provenance, fingerprints, validation evidence, and research results.

#### ***What you cannot do in this build***

- Submit live brokerage orders.
- Treat a scrubbed source as automatically live-authorized.
- Treat synthetic smoke-test results as trading performance.
- Bypass causal-integrity rejection.
- Use future bars or future completed higher-timeframe candles to influence a past signal.

### 2. Authority and safety model

QuantForge uses separate authority boundaries. Source scrubbing establishes what code appears capable of doing; it does not authorize live trading. Research approval establishes permission to execute within the governed research environment only.

Live trading status for this build: DISABLED.

### 3. Installation and first run

1. Obtain the current stable build package and preserve its SHA-256 hash.
2. Extract the package into a dedicated QuantForge directory.
3. Review the included release manifest and this manual before first use.
4. Confirm the environment prerequisites documented by the release manifest.
5. Start QuantForge in research/simulation mode.
6. Confirm that live brokerage/order authority is shown as disabled.
7. Run the built-in validation/release checks available in the environment.
8. Begin with a known test strategy or the supplied quarantined examples.

Native Android/Windows runtime installation is not represented as fully validated until native builds and device/runtime validation are completed.

### 4. Importing a strategy or indicator

Use this sequence:

Quarantine → fingerprint → static scrub → feature inventory → authority classification → user feature selection → sanitized research adapter → regression tests → provenance registration → research

The original source remains preserved separately from sanitized or adapted versions.

#### ***Before approving an import***

Check:

- file type and source identity;
- network or external-system access;
- file/process/DLL/credential behavior;
- order-entry and account-control behavior;
- data-source behavior;
- time/session assumptions;
- indicator lookback and future-data risks;
- execution timing assumptions.

### 5. Current imported research queue

#### ***ZeroLagLiquidity***

NinjaTrader strategy candidate. Requires cleanup of source-export corruption and causal execution wrapping. Its documentation describes a one-micro, intraday MES/MNQ profile, but those parameters are source documentation and are not themselves proof of performance.

#### ***VWAP Mean Reversion***

Python research candidate. Uses session VWAP and standard-deviation bands. Its original implementation assigns a same-bar close-based entry; QuantForge must use an explicit close-auction model or a subsequent-bar fill to prevent ambiguous execution timing.

## ***Volume Sentiment Breakout Channels***

Pine indicator candidate. It is treated as an indicator/signal provider rather than an executable strategy until a controlled strategy conversion is separately approved.

## ***TrendRebalanceMapHerman***

NinjaTrader strategy candidate. Its directional condition requires review/confirmation before optimization; QuantForge must preserve source semantics rather than silently correcting intent.

## ***Ultimate Strategy Template artifacts***

Reference/template material. The supplied text artifact does not establish that every feature described in the reference PDF is implemented in the supplied executable artifact. Complete executable source is required for executable import.

## **6. Research workflow**

Use the following progression:

1. Static audit — understand source behavior.
2. Causal backtest — use chronological historical data and an explicit fill model.
3. Walk-forward validation — separate development and validation periods.
4. Replay — reproduce event-by-event behavior.
5. Simulated forward test — use historical-forward or simulated real-time data without live order authority.
6. Evidence review — inspect orders, fills, positions, equity, risk, provenance, and model fingerprints.

## **7. Causal integrity**

At observation time  $T$ , only information available at or before  $T$  may influence a signal. Future bars, future indicator values, and future completed higher-timeframe candles are prohibited. A causal-integrity violation is a hard rejection.

A signal based on a completed current bar cannot silently receive a same-bar fill. Same-bar execution requires an explicit close-auction model; otherwise use the next-bar-open or next-eligible-tick model.

## **8. Multi-strategy forward testing**

Each strategy receives independent feed state, strategy state, simulated account state, ledger namespace, evidence chain, and model fingerprint. Duplicate IDs, invalid capability manifests, fingerprint mismatches, and cross-strategy namespace conflicts are rejected.

## **9. Backtest results: what counts as valid**

A performance result requires appropriate historical market data plus instrument-specific tick size, point value, session calendar/timezone, commissions, slippage/fill model, parameters, and defined development/validation/forward periods.

QuantForge must not invent win rate, net profit, profit factor, Sharpe ratio, drawdown, trade count, or other performance metrics when the necessary data is absent.

Smoke tests prove software behavior. They are not evidence of trading performance.

## 10. Understanding results

Review at minimum:

- trade/order sequence;
- entry and exit timestamps;
- fill prices and fill model;
- position state;
- realized/unrealized P&L;
- drawdown and risk exposure;
- fees and slippage;
- rejected or skipped signals;
- causal-integrity status;
- strategy/model fingerprints;
- data and session assumptions.

Do not judge a strategy from a single headline metric. Review the underlying ledger and validation period.

## 11. Troubleshooting

Import rejected: review the scrub report and capability manifest; do not bypass the rejection.

Causal violation: identify the exact future-data dependency or timing ambiguity and correct the adapter/source before retesting.

Unexpected fill: inspect the configured fill model and event timestamp. Same-bar fills require an explicit close-auction model.

Multiple strategies interfere: verify unique strategy IDs and isolated ledger/feed namespaces.

No performance report: verify that valid historical market data and all instrument/execution parameters were supplied.

Native build failure: record the compiler/SDK/device environment and preserve the failure evidence; native validation is separate from source-level structural checks.

## 12. Production-value operating checklist

Before research:

- [ ] Correct build and hash recorded.
- [ ] Research-only mode confirmed.
- [ ] Source quarantined and fingerprinted.
- [ ] Scrub report reviewed.
- [ ] Capabilities explicitly approved.
- [ ] Causal/execution policy selected.
- [ ] Market data and instrument assumptions verified.

Before forward testing:

- [ ] Backtest completed without causal violations.
- [ ] Walk-forward validation completed.
- [ ] Replay behavior checked.
- [ ] Simulated account and ledger isolated.
- [ ] Live authority confirmed disabled.

## 13. Glossary

Causal integrity: ensuring a historical decision only uses information available at that historical moment.

Quarantine: isolated staging area where imported source is inspected before research execution.

Capability manifest: machine-readable description of what an imported component is permitted to access or do.

Forward test: testing a strategy sequentially as if time were moving forward, without allowing future information to influence earlier decisions.

Close-auction model: an explicitly defined model for allowing an order generated from a completed bar to be filled at that bar's close.

Ledger: chronological record of orders, fills, positions, P&L, fees, and risk state.

## 14. Current release limitations

Native .NET compilation and native Android/Windows device execution are not verified in the current environment. Live brokerage and live order submission remain disabled.

## 15. Release history

v25.75 — import provenance hardening.

v25.80 — execution timing integrity.

v25.85 — multi-strategy ledger isolation.

v25.90 — stable release gate.

v25.95 — consumer-manual information architecture audit.

v26.00 — first-run and operator workflow hardening.

v26.05 — production-value manual, PDF manual, usability audit, and release documentation consolidation.

## v26.15–v26.25 Data Readiness Update

QuantForge now treats user-supplied NinjaTrader exports as immutable research evidence. Source files are fingerprinted before admission, and missing coverage is never silently filled.

### ***Current MES minute dataset***

The current local intake contains 25 NT8 `Last` exports and 2,181,806 parsed rows. Basic structural validation found zero malformed rows, duplicate timestamps, out-of-order timestamps, invalid OHLC relationships, negative volume rows, or rows outside the 0.25-point price grid in the current files.

Three coverage gaps remain: Jun→Sep 2023, Jun→Sep 2024, and Jun→Sep 2025. These correspond to the missing Sep-23, Sep-24, and Sep-25 exports.

### ***What happens when the missing data arrives***

1. Preserve the original files.
2. Fingerprint every file.
3. Re-run the complete parser/integrity checks.
4. Verify contract coverage and document rollover/continuity rather than silently repairing it.
5. Admit the canonical dataset only after the data gate passes.
6. Run backtests using causal execution timing.
7. Run bounded optimization with separated development, validation, and untouched forward periods.
8. Publish consolidated reports under `strategy optimization/`.

### ***Important limitation***

No current performance figure is a validated six-year result. QuantForge intentionally blocks optimization until the missing source coverage is supplied.

### **Data safety rules**

- Original source exports are never overwritten.
- Silent bar creation or repair is prohibited.
- Forward-looking information is prohibited.
- Same-bar fills require an explicitly declared execution model.
- Incomplete datasets cannot be presented as complete performance evidence.

### **v26.30 update — External data sources**

QuantForge can record a shared repository or cloud-folder location as provenance, but a link does not make data usable. The system must retrieve the actual bytes, fingerprint them, validate their structure and chronology, and admit them before they can affect research results.

This prevents inaccessible or partially retrieved external datasets from silently entering a backtest.

### **v26.35 update — Research run identity**

Every publishable backtest or optimization result must be tied to immutable identities for the dataset, strategy, execution policy, parameter set, and temporal partition. Missing identity, failed causal validation, or missing optimization admission prevents publication as validated performance.